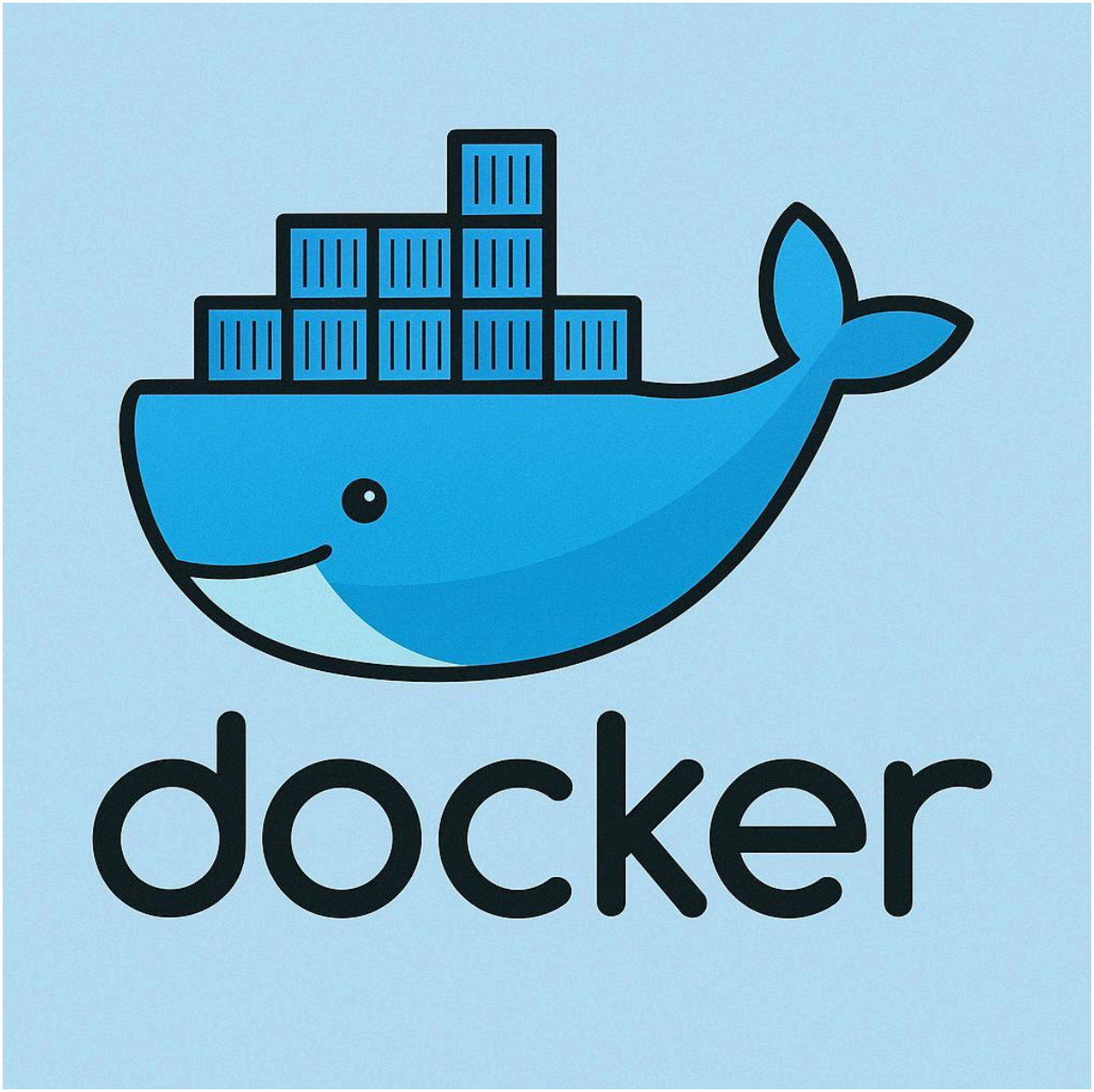
Day 14 — Docker Fundamentals



What is Docker?

Imagine you've built a Python application on your computer. It works perfectly there, but when someone runs it on another computer, they may have different Python versions or missing packages.

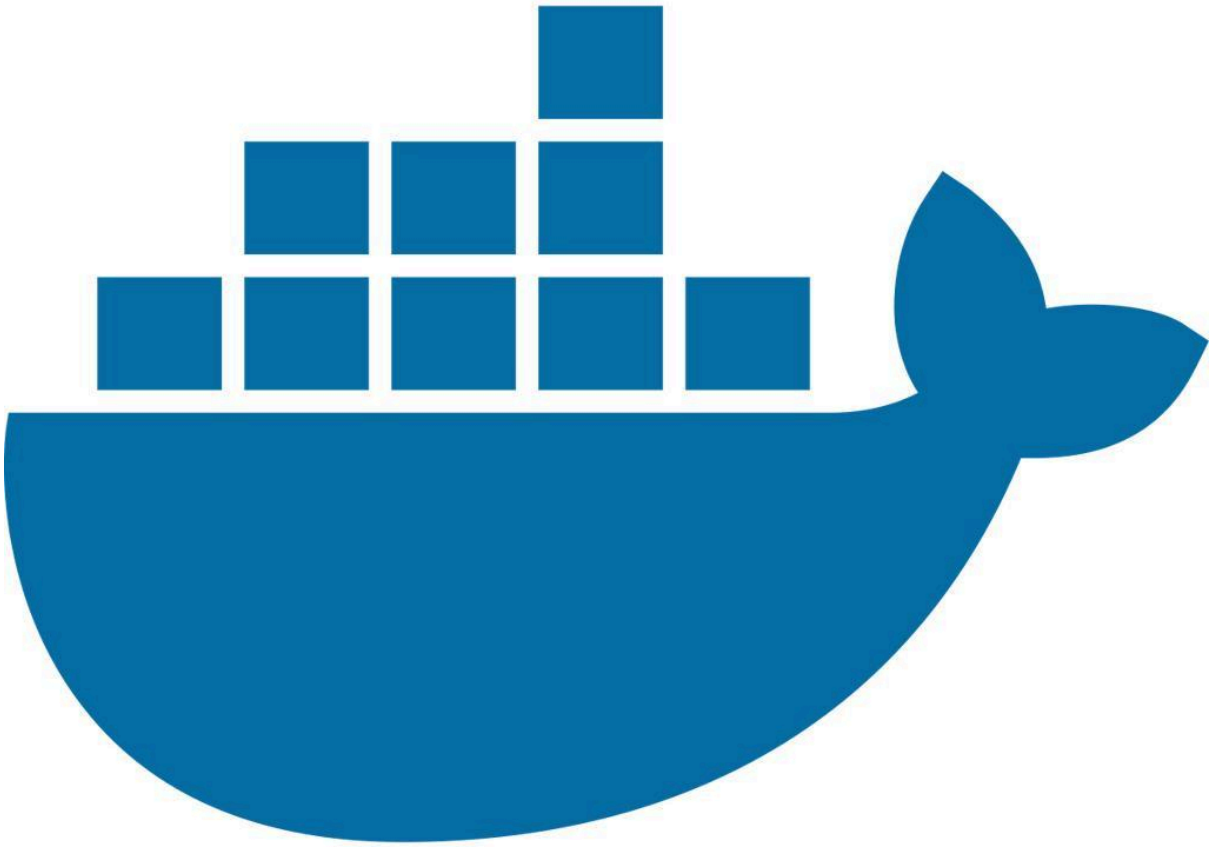
Docker helps package an application with the environment and dependencies it needs so it can run more consistently on different systems.

For example, your FastAPI application may need:

- Python
- FastAPI
- Uvicorn
- Your application code

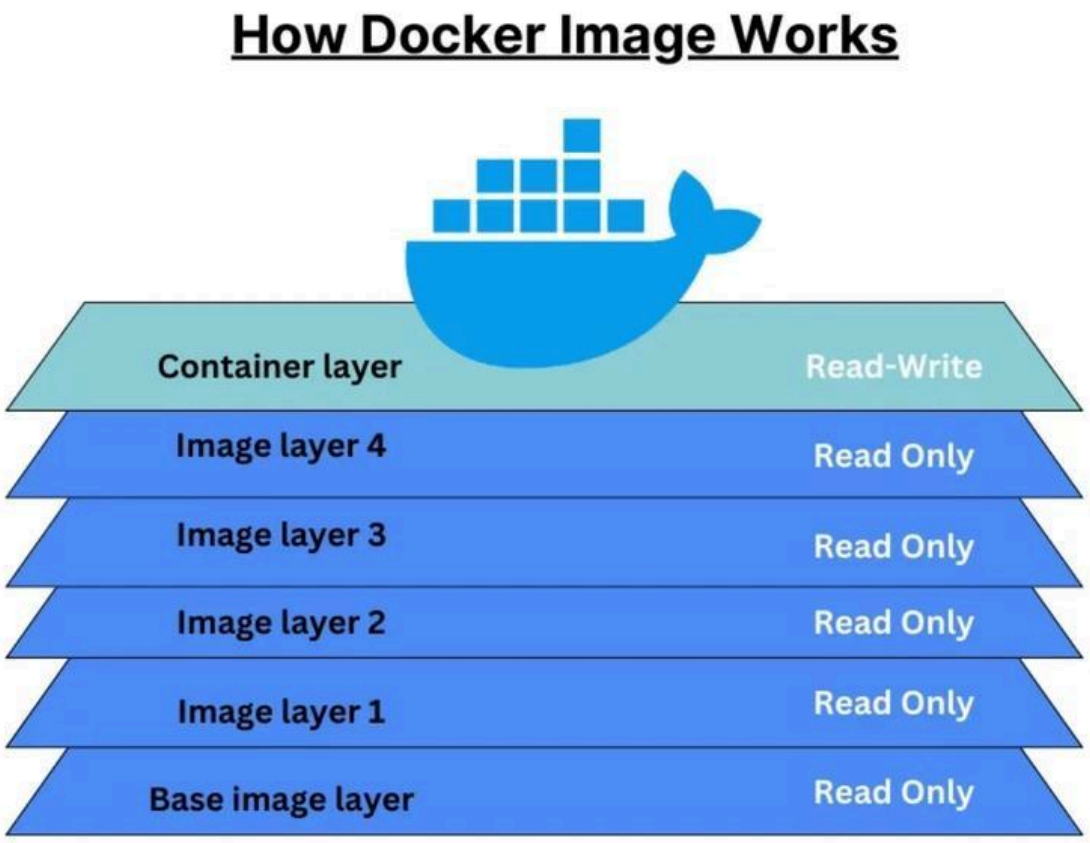
Docker lets you define these requirements and run the application inside a container.

five terms



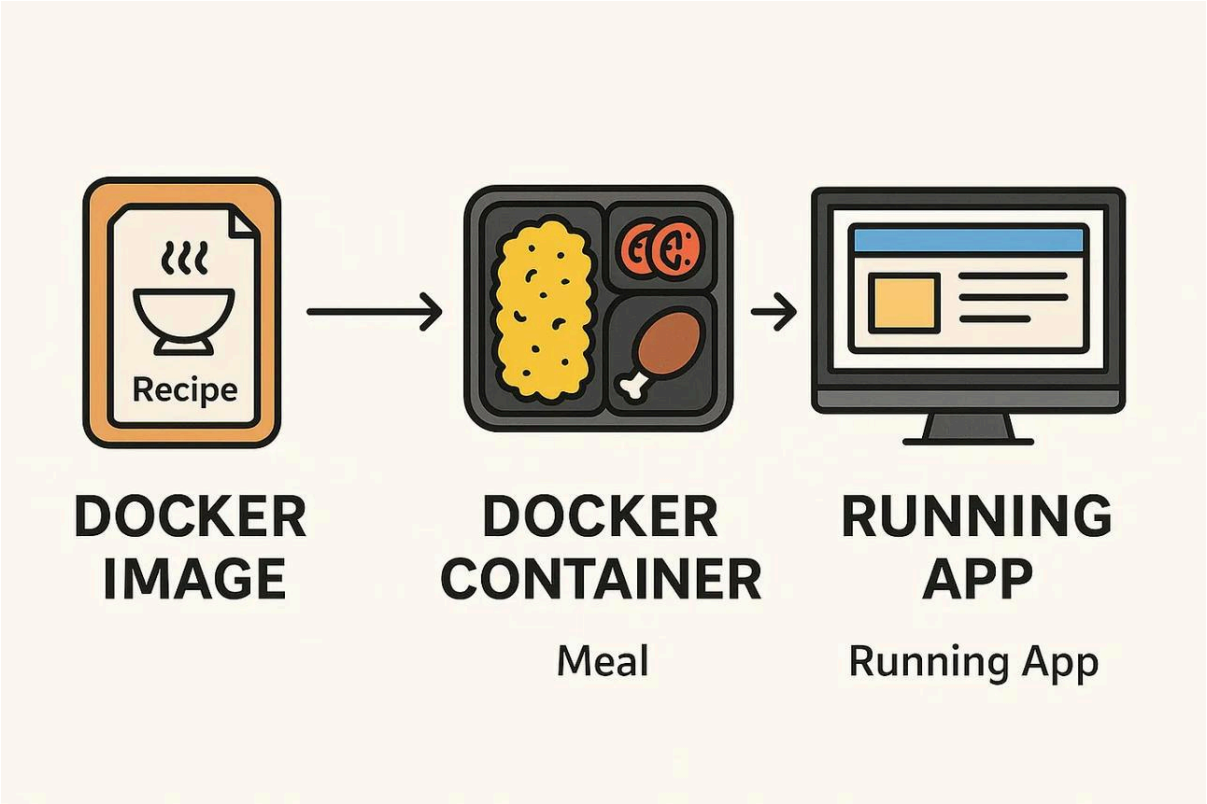
Docker

A platform and set of tools used to build, distribute and run applications in containers.



Docker image

A packaged template containing the application, required files and instructions to run it. You create containers from images.



Container

A running instance of an image. Think of an image as a recipe and a container as the running application made from that recipe.



```dockerfile
FROM python:3.11.9-slim-bullseye

WORKDIR /app

# Copy only requirements first to leverage Docker cache on dependencies
COPY requirements.txt .

# Install dependencies without cache for smaller image
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of the app code
COPY . .

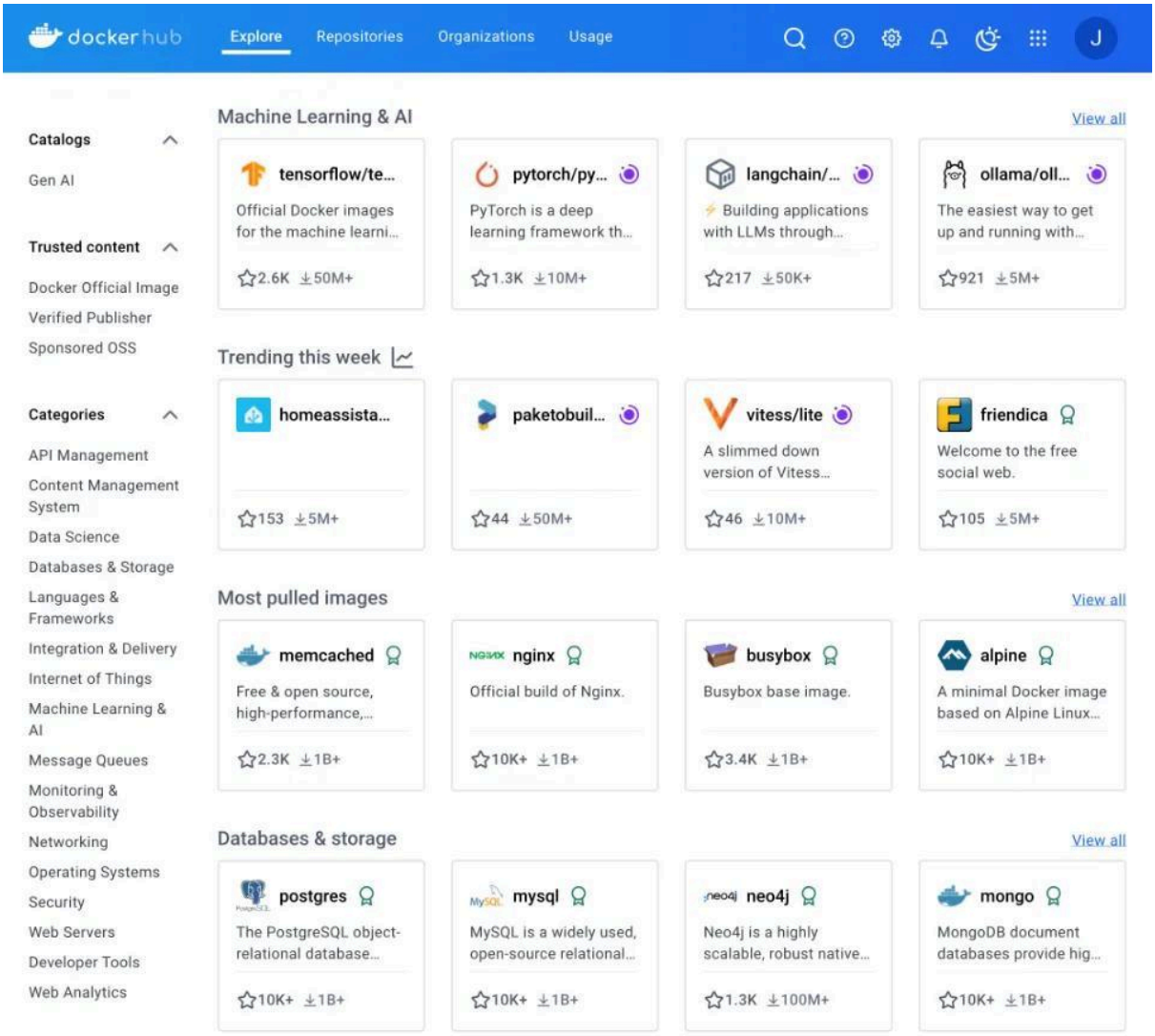
# Expose port
EXPOSE 8000

# Start the FastAPI app using uvicorn
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Dockerfile

A text file containing instructions Docker uses to build an image.



Docker Hub

An online registry where people publish and download container images.

Step 1 — Check whether Docker is installed

Since you're using VS Code and Ubuntu through WSL, open your Ubuntu terminal.

Run:

docker --version

This displays the installed Docker version if the Docker CLI is available.

Then run:

docker info

This displays information about the Docker installation and engine. If you get a daemon connection error, Docker may not be running or WSL integration may not be configured.

Finally, run:

docker run hello-world

This downloads and runs a small test image if necessary. A successful run displays a message confirming Docker can run a container.

Do not install or change settings yet if a command fails. Send me the exact error or a screenshot first, and we'll fix it step by step.

Essential Docker commands

Command	Meaning
`docker --version`	Shows the Docker CLI version
`docker info`	Shows Docker engine information
`docker images`	Lists downloaded images
`docker ps`	Lists running containers
`docker ps -a`	Lists all containers, including stopped ones
`docker pull python:3.12-slim`	Downloads a Python image
`docker run hello-world`	Runs the Docker test container

`docker logs CONTAINER_ID`	Displays a container's logs
`docker stop CONTAINER_ID`	Stops a running container
`docker rm CONTAINER_ID`	Removes a stopped container
`docker rmi IMAGE_NAME`	Removes an image, if it is not in use

Replace `CONTAINER_ID` with the actual ID or name shown by Docker.

Practise these first

docker --version
docker images
docker ps
docker ps -a

Understand the output of each command before continuing.

Step 2 — Create your first Docker project

We'll make a small Python application that prints a message. This demonstrates how a Docker image can package and run Python code.

Create a folder inside your `Project prep` directory:

mkdir -p day14-docker
cd day14-docker

Create a Python file:

touch app.py

Open `day14-docker/app.py` in VS Code and add:

print("Hello from my first Docker application!")print("Learning Docker for my Syntova project.")

Save the file.

Create another file named `Dockerfile` — exactly that name, with no `.txt` extension.

Paste this:

FROM python:3.12-slim

WORKDIR /app

COPY app.py .

CMD ["python", "app.py"]

Instruction	Meaning
`FROM python:3.12-slim`	Uses a lightweight Python 3.12 base image
`WORKDIR /app`	Sets the working directory inside the image
`COPY app.py .`	Copies `app.py` into the working directory
`CMD ["python", "app.py"]`	Runs the Python application when the container starts

Step 3 — Build and run the image

Make sure your terminal is inside `day14-docker` and Docker is running.

Build an image:

docker build -t my-first-python-app .

Meaning:

- `docker build` builds an image.
- `-t my-first-python-app` gives it a name.
- `.` tells Docker to use the current directory as the build context.

Check the image:

docker images

Now run it:

docker run --name python-demo my-first-python-app

Expected output:

Hello from my first Docker application!
Learning Docker for my Syntova project.

Because this is a short script, the container exits after printing the messages. That is normal.

Check the container:

docker ps -a

You should see the `python-demo` container, likely with an `Exited` status.

Remove the stopped container if you no longer need it:

docker rm python-demo

You can keep the image for further practice.

Step 4 — Dockerise your FastAPI application

Once the Python example works, you can practise with a web API similar to the one you built on Day 9.

Create a separate folder named `day14-fastapi-docker`. Inside it, create these two files.

`main.py`

from fastapi import FastAPIapp = FastAPI()@app.get("/")def home(): return {"message": "My FastAPI app is running in Docker"}@app.get("/hello")def hello(): return {"message": "Hello from a Docker container"}

`requirements.txt`

fastapi
uvicorn

`Dockerfile`

FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY main.py .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

What is new in this Dockerfile?

- `COPY requirements.txt .` copies the dependency list.
- `RUN pip install ...` installs the required Python packages during image building.
- `COPY main.py .` copies your API code.
- `EXPOSE 8000` documents the port used by the application.
- `--host 0.0.0.0` lets the server accept connections through the container's network interface.

Build the image:

docker build -t syntova-fastapi .

Run the container:

docker run --name syntova-api -p 8000:8000 syntova-fastapi

Keep this terminal open. In your browser, visit:

- `http://localhost:8000`
- `http://localhost:8000/hello`
- `http://localhost:8000/docs`

The `-p 8000:8000` option maps port 8000 on your computer to port 8000 inside the container.

To stop the application, press Ctrl+C in the terminal. If the container remains stopped, you can remove it with:

docker rm syntova-api

Only try this section after your Docker installation is working and you've understood the basic Python example.

Interview questions with answers

Q1. What is Docker?

Docker is a platform for building, distributing and running applications in containers.

Q2. What is the difference between an image and a container?

An image is a template. A container is an instance created from that image.

Q3. What is a Dockerfile?

A text file containing instructions to build a Docker image.

Q4. What is the purpose of `docker build`?

It builds an image from a Dockerfile and its build context.

Q5. What does `docker run` do?

It creates and starts a container from an image.

Q6. What is the purpose of `-p 8000:8000`?

It maps host port 8000 to container port 8000, allowing you to access the application from your computer.

Q7. What is the difference between `docker ps` and `docker ps -a`?

`docker ps` lists running containers. `docker ps -a` also lists stopped containers.

Q8. Why use Docker for a FastAPI project?

Docker packages the application and its dependencies into a consistent environment, making it easier to run on other machines.

Today's assignment

Create a project called `day14-docker` containing:

```
day14-docker/
├── app.py
├── Dockerfile
└── README.md
```

README template

```
# Day 14: Docker Fundamentals

## Objective
Learn how to package and run a Python application using Docker.
```

Files
- app.py: Python application.
- Dockerfile: Instructions for building the image.

Commands Practised
- docker --version
- docker images
- docker ps
- docker build
- docker run
- docker ps -a

Command reference

Command	Meaning
`docker --version`	Displays Docker CLI version
`docker info`	Shows Docker engine information
`docker run hello-world`	Tests whether Docker can run a container
`docker images`	Lists local images
`docker ps`	Lists running containers
`docker ps -a`	Lists all containers
`docker pull IMAGE`	Downloads an image
`docker build -t NAME .`	Builds an image and gives it a name

`docker run NAME`	Creates and starts a container
`docker run -p 8000:8000 NAME`	Maps host port 8000 to container port 8000
`docker logs NAME`	Shows container logs
`docker stop NAME`	Stops a running container
`docker rm NAME`	Removes a container
`docker rmi IMAGE`	Removes an image if it is not in use